Client Script & UI Policy Implementation

Incident Management Form Customization on ServiceNow

Item	Detail
Project	Incident Form: Client Script and UI Policy
Platform	ServiceNow
Target table	Incident [incident]
Deliverables	4 Client Scripts + 1 VIP Client Script, 4 UI Policies, test plan, deployment guide
Document date	October 30, 2026

Table of Contents

1. Introduction and Objectives

2. Business Requirements

3. Prerequisites and Environment Setup

4. Core Concepts and Execution Order

5. Client Script Implementation

6. UI Policy Implementation

7. g_form API Quick Reference

8. Testing Plan

9. Troubleshooting Guide

10. Best Practices

11. Deployment with Update Sets

12. Conclusion and Future Enhancements

Appendix A: Incident State Values

1. Introduction and Objectives

The Incident form is the main workspace for service desk agents. Poorly guided forms lead to incomplete tickets, wrong categorization, and slow resolution. This project uses two client-side tools in ServiceNow, Client Scripts and UI Policies, to make the Incident form smarter, safer, and easier to use.

Client Scripts run JavaScript in the browser when the form loads, when a field changes, when the form is submitted, or when a list cell is edited. UI Policies provide a no-code (or low-code) way to make fields mandatory, visible, or read-only based on conditions.

Objectives

• Prevent edits to Closed incidents so history stays accurate.

• Force agents to capture resolution details before an incident is marked Resolved.

• Keep Category and Subcategory consistent by clearing stale subcategory values.

• Alert agents when a Priority 1 incident is raised or when the caller is a VIP.

• Reject poor-quality Short descriptions before the record is saved.

• Show only the fields that matter for the situation, such as the Configuration Item for Hardware issues.

Expected Outcomes

• Higher data quality on all new and updated incidents.

• Fewer reopened incidents caused by missing close notes.

• Faster handling of critical incidents through visible alerts.

• A repeatable, documented pattern the team can reuse on other tables.

2. Business Requirements

The table below lists each requirement, the tool chosen to implement it, and the reason for that choice.

ID	Requirement	Tool	Reason
BR-01	Closed incidents must be read-only	Client Script (onLoad)	Needs to lock several fields at once with a single message
BR-02	Subcategory must reset when Category changes	Client Script (onChange)	Requires clearing a value, which UI Policy cannot do alone
BR-03	Warn agents on Priority 1	Client Script (onChange)	Custom message on a specific value
BR-04	Short description must be meaningful	Client Script (onSubmit)	Validation that can stop form submission
BR-05	Warn when caller is a VIP	Client Script (onChange)	Needs a reference lookup on the caller record
BR-06	Close code and close notes required when Resolved	UI Policy	Simple mandatory rule driven by a condition
BR-07	Configuration Item required for Hardware	UI Policy	Simple mandatory rule driven by a condition
BR-08	On hold reason required when On Hold	UI Policy	Simple mandatory rule driven by a condition
BR-09	Guide assignment when In Progress	UI Policy (scripted)	Mandatory field plus an info message that clears when no longer true

Assumptions

• The instance uses the standard Incident table and out-of-the-box fields.

• Choice values for State follow the defaults listed in Appendix A.

• Changes are built and tested in a development or sub-production instance first.

3. Prerequisites and Environment Setup

Before building anything, confirm the following items are in place.

Access and Roles

• **admin** role, or a delegated developer role that can create Client Scripts and UI Policies.

• Access to a non-production instance for development and testing.

• At least one test user with the **itil** role to verify agent behavior.

• A test caller flagged as VIP to verify BR-05.

Create an Update Set

An Update Set captures every change you make so it can be moved to another instance.

1. Open **System Update Sets > Local Update Sets** and click **New**.

2. Name it **Incident Client Script and UI Policy v1**.

3. Add a short description listing the business requirements covered.

4. Click **Submit and Make Current**.

Verify the Form Layout

Open any incident and use **Form Layout** (or Form Builder) to confirm that Category, Subcategory, Priority, State, Close code, Close notes, Configuration item, On hold reason, Assignment group, and Assigned to are all on the form. Fields that are not on the form cannot be controlled by Client Scripts or UI Policies.

Test Data

Item	Purpose
Closed incident	Verifies BR-01
New incident	Used to change Category, Priority, State and Caller
VIP caller record	Verifies BR-05
Hardware incident	Verifies BR-07

4. Core Concepts and Execution Order

Client Script Types

Type	When it runs	Typical use
onLoad	After the form finishes loading	Set initial field states, show messages
onChange	When a specific field value changes	React to user input, clear dependent fields
onSubmit	When the user saves or submits the form	Validate data, block submission by returning false
onCellEdit	When a value is edited in a list	Validate inline list edits

UI Policy vs Client Script

Aspect	UI Policy	Client Script
Code required	No (optional scripts)	Yes
Mandatory / visible / read-only	Built-in actions	Done with g_form methods
Reverses automatically	Yes, with Reverse if false	No, you must code both cases
Custom validation	Limited	Full control
Maintenance effort	Low	Medium

Order of Execution

When a form loads, the platform runs onLoad Client Scripts first and then evaluates UI Policies. This is why a UI Policy can override a field state set by an onLoad Client Script. If a rule seems to be ignored, check whether another tool is changing the same field afterward.

1. Form loads and data is retrieved.

2. onLoad Client Scripts run.

3. UI Policies are evaluated and their actions applied.

4. The user edits fields: onChange Client Scripts and UI Policies re-evaluate.

5. The user saves: onSubmit Client Scripts run, then the server processes the record.

Important: client-side logic runs in the browser and can be bypassed through imports, APIs, or integrations. Treat it as user guidance, and back up critical rules with server-side Business Rules or Data Policies.

5. Client Script Implementation

Navigate to **System Definition > Client Scripts** and click **New**. For every script below, set Table to **Incident [incident]** and leave Active checked. Leave Global checked unless you want to limit the script to a specific view.

5.1 Lock Closed Incidents (onLoad) - BR-01

Field	Value
Name	Incident - Lock Closed Form
Type	onLoad
UI Type	All
Active	True

```
function onLoad() {
    // State 7 = Closed
    if (g_form.getValue('state') == '7') {
        var fields = ['short_description', 'description', 'category',
                      'subcategory', 'impact', 'urgency', 'assignment_group'];
        fields.forEach(function (f) {
            g_form.setReadOnly(f, true);
        });
        g_form.addInfoMessage('This incident is closed and cannot be edited.');
    }
}
```

How it works: The script reads the State value when the form opens. If it equals 7 (Closed), it loops through a list of field names and marks each read-only, then shows an info banner. Adding or removing fields only requires editing the array.

5.2 Clear Subcategory on Category Change (onChange) - BR-02

Field	Value
Name	Incident - Clear Subcategory
Type	onChange
Field name	Category
Active	True

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    g_form.clearValue('subcategory');
}
```

How it works: The isLoading check stops the script from running while the form is first populated, which would wipe saved data. Without this guard, every incident would lose its subcategory on open.

5.3 Warn on Priority 1 (onChange) - BR-03

Field	Value
Name	Incident - P1 Warning
Type	onChange
Field name	Priority
Active	True

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if (newValue == '1') {
        g_form.addInfoMessage('Priority 1 incident: notify the Major Incident Manager.');
    }
}
```

How it works: Priority is calculated from Impact and Urgency, and the form updates the Priority field automatically. This onChange fires when that value becomes 1 (Critical) and displays a reminder at the top of the form.

5.4 Validate Short Description (onSubmit) - BR-04

Field	Value
Name	Incident - Validate Short Description
Type	onSubmit
Active	True

```
function onSubmit() {
    var shortDesc = g_form.getValue('short_description').trim();
    if (shortDesc.length < 10) {
        g_form.showFieldMsg('short_description',
            'Short description must be at least 10 characters.', 'error');
        return false; // stops submission
    }
    return true;
}
```

How it works: Returning false from an onSubmit script cancels the save. The field message appears directly under the Short description field so the agent knows exactly what to fix. The minimum length of 10 can be adjusted to your standard.

5.5 VIP Caller Alert (onChange) - BR-05

Field	Value
Name	Incident - VIP Caller Alert
Type	onChange

Field	Value
Field name	Caller
Active	True

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    // Asynchronous lookup of the caller record
    g_form.getReference('caller_id', function (caller) {
        if (caller && caller.vip == 'true') {
            g_form.addInfoMessage('The caller is a VIP. Handle with priority.');
        }
    });
}
```

How it works: getReference retrieves the full User record for the selected caller. Always pass a callback function so the lookup is asynchronous and does not freeze the browser. The VIP flag is read from the returned record.

6. UI Policy Implementation

Navigate to **System UI > UI Policies** and click **New**. Set Table to **Incident [incident]**. After saving each policy, use the **UI Policy Actions** related list at the bottom to add the field actions. Tick **Reverse if false** on every policy so fields return to normal when the condition stops being true.

6.1 Resolution Fields Mandatory - BR-06

Setting	Value
Short description	Incident - Resolution fields mandatory
Condition	State is Resolved
Reverse if false	True
On load	True

Field	Mandatory	Visible	Read only
Close code	True	True	Leave alone
Close notes	True	True	Leave alone

6.2 Show Configuration Item for Hardware - BR-07

Setting	Value
Short description	Incident - Show CI for Hardware
Condition	Category is Hardware
Reverse if false	True
On load	True

Field	Mandatory	Visible	Read only
Configuration item	True	True	Leave alone

6.3 Require Reason When On Hold - BR-08

Setting	Value
Short description	Incident - On Hold reason
Condition	State is On Hold
Reverse if false	True
On load	True

Field	Mandatory	Visible	Read only
On hold reason	True	True	Leave alone

6.4 Guide Assignment When In Progress (Scripted) - BR-09

This policy combines a mandatory field with scripts. To use scripts, tick **Run scripts** on the UI Policy form, which reveals the Execute if true and Execute if false boxes.

Setting	Value
Short description	Incident - Assignment guidance
Condition	State is In Progress
Reverse if false	True
Run scripts	True

Field	Mandatory	Visible	Read only
Assignment group	True	True	Leave alone

Execute if true

```
function onCondition() {
    g_form.showFieldMsg('assigned_to',
        'Assign this incident to yourself or a team member.', 'info');
}
```

Execute if false

```
function onCondition() {
    g_form.hideFieldMsg('assigned_to', true);
}
```

How it works: When State becomes In Progress, the Assignment group becomes mandatory and a hint appears under Assigned to. When the state changes away, the Execute if false script removes the hint so the form returns to its normal appearance.

Policy Order

If two UI Policies affect the same field, the policy with the higher **Order** value runs last and wins. Keep Order values at the default of 100 unless you have a specific conflict to resolve.

7. g_form API Quick Reference

The g_form object is the main client-side API for working with form fields. These are the methods used in this project, plus a few related ones you are likely to need.

Method	Purpose
g_form.getValue(field)	Returns the current value of a field
g_form.setValue(field, value)	Sets a field value
g_form.clearValue(field)	Clears a field value
g_form.setReadOnly(field, bool)	Makes a field read-only or editable
g_form.setMandatory(field, bool)	Makes a field mandatory or optional
g_form.setDisplay(field, bool)	Shows or hides a field and reclaims space
g_form.setVisible(field, bool)	Shows or hides a field and leaves space
g_form.addInfoMessage(text)	Shows a blue info banner at the top of the form
g_form.addErrorMessage(text)	Shows a red error banner at the top of the form
g_form.showFieldMsg(field, text, type)	Shows a message under a field (info, error)
g_form.hideFieldMsg(field, clear)	Removes messages under a field
g_form.getReference(field, callback)	Fetches the referenced record asynchronously
g_form.isNewRecord()	Returns true if the record has not been saved

Client Script Parameters

Parameter	Meaning
control	The HTML element of the changed field
oldValue	Value before the change
newValue	Value after the change
isLoading	True while the form is loading. Use it to skip onChange logic during load
isTemplate	True when the change comes from applying a template

8. Testing Plan

Run each test in the development instance while logged in as a user with the itil role. Record the actual result and mark Pass or Fail.

#	Requirement	Steps	Expected result	Result
1	BR-01	Open a Closed incident	Listed fields are read-only; info message shown	
2	BR-02	Select a Category, pick a Subcategory, then change the Category	Subcategory clears	
3	BR-02	Open an existing incident with a saved Subcategory	Subcategory is NOT cleared on load	
4	BR-03	Set Impact and Urgency to High so Priority becomes 1	P1 warning appears	
5	BR-04	Enter a 5-character Short description and save	Error under field; save blocked	
6	BR-04	Enter a 15-character Short description and save	Record saves	
7	BR-05	Select a VIP caller	VIP info message appears	
8	BR-05	Select a non-VIP caller	No VIP message	
9	BR-06	Set State to Resolved	Close code and Close notes mandatory	
10	BR-06	Change State back to In Progress	Close fields no longer mandatory	
11	BR-07	Set Category to Hardware	Configuration item mandatory	
12	BR-08	Set State to On Hold	On hold reason mandatory	
13	BR-09	Set State to In Progress	Assignment group mandatory; hint under Assigned to	
14	BR-09	Change State to On Hold	Hint removed	
15	All	Repeat key tests on the mobile and Service Operations Workspace views	Behavior is consistent or limitations documented	

Regression Checks

• Create a new incident end to end and confirm there are no console errors.

• Open the browser developer console (F12) and verify no JavaScript errors appear on form load.

• Confirm that existing incidents open at normal speed.

9. Troubleshooting Guide

Symptom	Likely cause	Fix
Script does not run at all	Script is inactive, or wrong table or field selected	Check Active, Table and Field name on the Client Script record
onChange runs when the form opens	Missing isLoading check	Add: if (isLoading) return;
Field rule ignored	A UI Policy or another script changes the same field afterward	Review execution order; check other policies and scripts on the table
Message does not appear	Wrong field name used	Use the field backend name (column name), not the label
Form freezes briefly	Synchronous server call such as getReference without callback	Always pass a callback to getReference
UI Policy does not reverse	Reverse if false is unchecked	Enable Reverse if false
Script works in classic UI but not in workspace	UI Type is set to Desktop only	Set UI Type to All, or create a workspace-compatible version
Closed form still editable	State value differs from 7 on this instance	Verify Closed choice value in the State dictionary entry

Debugging Tips

• Use **jslog()** inside a Client Script to write messages to the Script Debugger or JavaScript log.

• Use the browser developer console to see script errors in real time.

• Use **System Diagnostics > Session Debug** to trace UI Policy execution.

• Test with a second user role to rule out ACL or role-based differences.

10. Best Practices

• **Prefer UI Policies for simple rules.** They are easier to maintain, can be edited without code, and reverse automatically.

• **Always guard onChange scripts** with isLoading and empty-value checks.

• **Never use synchronous server calls** from the client. Use callbacks or GlideAjax.

• **Avoid DOM manipulation.** Use g_form methods instead; direct DOM edits break in workspaces and after upgrades.

• **Do not hard-code sys_ids.** Use field values or system properties.

• **Use clear naming:** Table - Purpose, such as "Incident - Lock Closed Form".

• **Add comments** to scripts explaining why the rule exists, not just what it does.

• **Keep scripts small.** One script per purpose is easier to test and disable.

• **Back up with server-side rules.** Use Business Rules or Data Policies for any rule that must never be bypassed.

• **Set UI Type correctly.** Choose All when the rule should apply on desktop, mobile, and workspace.

• **Document changes** in the Update Set description and keep this document up to date.

Performance Considerations

Every onLoad and onChange script adds work in the browser. Limit scripts to those that deliver real value, avoid loops over large datasets, and combine related field changes into a single script where it keeps the code readable.

11. Deployment with Update Sets

Once testing passes in development, move the changes to the next environment using the Update Set created in section 3.

1. In the development instance, open the Update Set and confirm every Client Script and UI Policy is listed under **Customer Updates**.

2. Set the Update Set State to **Complete**.

3. Go to **Update Set > Export to XML**, or retrieve it from the target instance using **Retrieve Update Sets**.

4. On the target instance, open **Retrieved Update Sets**, then click **Preview Update Set**.

5. Resolve any preview problems, such as skipped or conflicting updates.

6. Click **Commit Update Set**.

7. Repeat the testing plan in the target instance.

8. Promote to production during an approved change window.

Rollback Plan

• Deactivate the new Client Scripts and UI Policies by unticking Active. This is the fastest rollback.

• If needed, back out the Update Set using **Back Out** on the committed Update Set record.

• Communicate the change and the rollback status to service desk leads.

Change Management Checklist

Item	Done
Change request raised and approved	
Testing results recorded	
Service desk briefed on new behavior	
Rollback plan confirmed	
Documentation updated	

12. Conclusion and Future Enhancements

This project delivers a set of Client Scripts and UI Policies that make the Incident form more reliable and easier to use. Closed records are protected, resolution data is captured, critical and VIP incidents are highlighted, and agents receive immediate feedback when data is incomplete.

Future Enhancements

• Extend the same pattern to Problem, Change Request and Service Catalog forms.

• Add a server-side Business Rule or Data Policy to enforce close notes on import and API updates.

• Replace hard-coded values with system properties, such as the minimum Short description length.

• Use GlideAjax for lookups that require server logic, such as checking whether a Configuration Item has an open Change.

• Add Workspace-specific UI Actions or Declarative Actions where the classic Client Script approach is limited.

• Create automated tests with the Automated Test Framework (ATF) to cover the scenarios in the testing plan.

Appendix A: Incident State Values

Label	Value
New	1
In Progress	2
On Hold	3
Resolved	6
Closed	7
Canceled	8

Values shown are the out-of-the-box defaults. Confirm them on your instance under **System Definition > Dictionary** by opening the State field on the Incident table.